

ObjektinisProgramavimas-3

3.0

Sugeneruota Doxygen 1.17.0

1 Concept Index	1
1.1 Concepts	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Concept Documentation	7
4.1 VectorElement Concept Reference	7
4.1.1 Concept definition	7
4.1.2 Smulkus aprašymas	7
5 Klasės Dokumentacija	9
5.1 Vector< T > Klasė Šablonas	9
5.1.1 Smulkus aprašymas	13
5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	13
5.1.2.1 Vector() [1/5]	13
5.1.2.2 Vector() [2/5]	13
5.1.2.3 Vector() [3/5]	14
5.1.2.4 Vector() [4/5]	14
5.1.2.5 Vector() [5/5]	14
5.1.3 Metodų Dokumentacija	15
5.1.3.1 append_range()	15
5.1.3.2 assign() [1/3]	15
5.1.3.3 assign() [2/3]	15
5.1.3.4 assign() [3/3]	16
5.1.3.5 assign_range()	16
5.1.3.6 at() [1/2]	16
5.1.3.7 at() [2/2]	17
5.1.3.8 back() [1/2]	17
5.1.3.9 back() [2/2]	17
5.1.3.10 capacity()	17
5.1.3.11 clear()	18
5.1.3.12 data() [1/2]	18
5.1.3.13 data() [2/2]	18
5.1.3.14 emplace()	18
5.1.3.15 emplace_back()	19
5.1.3.16 empty()	19
5.1.3.17 erase() [1/2]	19
5.1.3.18 erase() [2/2]	20
5.1.3.19 front() [1/2]	20
5.1.3.20 front() [2/2]	20

5.1.3.21 get_allocator()	21
5.1.3.22 insert() [1/5]	21
5.1.3.23 insert() [2/5]	21
5.1.3.24 insert() [3/5]	22
5.1.3.25 insert() [4/5]	22
5.1.3.26 insert() [5/5]	23
5.1.3.27 insert_range()	23
5.1.3.28 max_size()	23
5.1.3.29 operator std::vector< T >()	24
5.1.3.30 operator<=>()	24
5.1.3.31 operator=() [1/3]	24
5.1.3.32 operator=() [2/3]	25
5.1.3.33 operator=() [3/3]	25
5.1.3.34 operator==()	25
5.1.3.35 operator[]() [1/2]	26
5.1.3.36 operator[]() [2/2]	26
5.1.3.37 pop_back()	26
5.1.3.38 push_back() [1/2]	26
5.1.3.39 push_back() [2/2]	27
5.1.3.40 reserve()	27
5.1.3.41 resize()	27
5.1.3.42 shrink_to_fit()	28
5.1.3.43 size()	28
5.1.3.44 swap()	28
6 Failo Dokumentacija	29
6.1 Vector.h Failo Nuoroda	29
6.1.1 Smulkus aprašymas	30
6.1.2 Funkcijos Dokumentacija	30
6.1.2.1 erase()	30
6.1.2.2 erase_if()	31
6.1.2.3 swap()	32
6.2 Vector.h	32
Rodyklė	39

skyrius 1

Concept Index

1.1 Concepts

Here is a list of all documented concepts with brief descriptions:

VectorElement	
Sąvoka, reikalaujanti, kad tipas <code>T</code> būtų perkeliamas (<code>std::movable</code>)	7

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Vector< T >	
Dinaminio masyvo šabloninė klasė	9

skyrius 3

Failo Indeksas

3.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

Vector.h	Dinaminio masyvo šabloninė klasė, analogiška <code>std::vector</code>	29
--------------------------	---	--------------------

skyrius 4

Concept Documentation

4.1 VectorElement Concept Reference

Sąvoka, reikalaujanti, kad tipas `T` būtų perkeliamas (`std::movable`).

```
#include <Vector.h>
```

4.1.1 Concept definition

```
template<typename T>  
concept VectorElement = std::movable<T>
```

4.1.2 Smulkus aprašymas

Sąvoka, reikalaujanti, kad tipas `T` būtų perkeliamas (`std::movable`).

Tik tokie tipai gali būti `Vector` elementais.

Template Parameters

<code>T</code>	Tikrinamas tipas.
----------------	-------------------

skyrius 5

Klasės Dokumentacija

5.1 Vector< T > Klasė Šablonas

Dinaminio masyvo šabloninė klasė.

```
#include <Vector.h>
```

Vieši Tipai

- using **value_type** = T
Elementų tipas.
- using **allocator_type** = std::allocator<T>
Atminties valdytojo tipas.
- using **size_type** = std::size_t
Dydžiui skirtas tipas be ženklo.
- using **difference_type** = std::ptrdiff_t
Atstumui tarp elementų skirtas tipas su ženklu.
- using **reference** = T&
Elemento nuorodos tipas.
- using **const_reference** = const T&
Konstantinės elemento nuorodos tipas.
- using **pointer** = typename std::allocator_traits<std::allocator<T>>::pointer
Rodyklės tipas.
- using **const_pointer** = typename std::allocator_traits<std::allocator<T>>::const_pointer
Konstantinės rodyklės tipas.
- using **iterator** = T*
Tiesioginio iteratoriaus tipas.
- using **const_iterator** = const T*
Konstantinio tiesioginio iteratoriaus tipas.
- using **reverse_iterator** = std::reverse_iterator<[iterator](#)>
Atvirkštinio iteratoriaus tipas.
- using **const_reverse_iterator** = std::reverse_iterator<[const_iterator](#)>
Konstantinio atvirkštinio iteratoriaus tipas.

Vieši Metodai

- **Vector** ()
Numatytasis konstruktorius. Sukuria tuščią masivą.
- **Vector** (size_type n, const T &val=T{})
Konstruktorius su pradiniu dydžiu ir reikšme.
- **Vector** (const Vector &other)
Kopijavimo konstruktorius.
- **Vector** (Vector &&other) noexcept
Perkėlimo konstruktorius.
- template<std::input_iterator InputIt>
Vector (InputIt first, InputIt last)
Konstruktorius iš iteratorių diapazono.
- **Vector** (std::initializer_list< T > il)
Konstruktorius iš inicializavimo sąrašo.
- ~**Vector** ()
Destruktorius. Sunaikina visus elementus ir atlaisvina atmintį.
- **Vector** & operator= (const Vector &other)
Kopijavimo priskyrimo operatorius.
- **Vector** & operator= (Vector &&other) noexcept
Perkėlimo priskyrimo operatorius.
- **Vector** & operator= (std::initializer_list< T > il)
Priskyrimo operatorius iš inicializavimo sąrašo.
- operator std::vector< T > () const
Netiesioginis konvertavimas į std::vector<T>.
- operator std::span< T > ()
Netiesioginis konvertavimas į kintamąjį std::span<T>.
- operator std::span< const T > () const
Netiesioginis konvertavimas į konstantinį std::span<const T>.
- template<std::input_iterator InputIt>
void assign (InputIt first, InputIt last)
Pakeičia turinį elementais iš iteratorių diapazono.
- void assign (size_type n, const value_type &val)
Pakeičia turinį n kopijų reikšmės val.
- void assign (std::initializer_list< value_type > il)
Pakeičia turinį inicializavimo sąrašo elementais.
- template<std::ranges::input_range R>
void assign_range (R &&r)
Pakeičia turinį diapazono elementais (C++23).
- allocator_type get_allocator () const noexcept
Grąžina naudojamą atminties valdytojo kopiją.
- reference operator[] (size_type n)
Prieiga prie elemento pagal indeksą (be ribų tikrinimo).
- const_reference operator[] (size_type n) const
Konstantinė prieiga prie elemento pagal indeksą (be ribų tikrinimo).
- reference at (size_type n)
Prieiga prie elemento pagal indeksą su ribų tikrinimu.
- const_reference at (size_type n) const
Konstantinė prieiga prie elemento pagal indeksą su ribų tikrinimu.
- reference front ()
Grąžina nuorodą į pirmą elementą.

- `const_reference front () const`
Grąžina konstantinę nuorodą į pirmą elementą.
- `reference back ()`
Grąžina nuorodą į paskutinį elementą.
- `const_reference back () const`
Grąžina konstantinę nuorodą į paskutinį elementą.
- `value_type * data () noexcept`
Tiesioginis prieigos prie vidinės atminties rodyklė.
- `const value_type * data () const noexcept`
Konstantinė rodyklė į vidinę atmintį.
- `iterator begin () noexcept`
Pradžios iteratorius.
- `const_iterator begin () const noexcept`
Konstantinis pradžios iteratorius.
- `const_iterator cbegin () const noexcept`
Konstantinis pradžios iteratorius (cbegin).
- `iterator end () noexcept`
Pabaigos iteratorius (rodo už paskutinio elemento).
- `const_iterator end () const noexcept`
Konstantinis pabaigos iteratorius.
- `const_iterator cend () const noexcept`
Konstantinis pabaigos iteratorius (cend).
- `reverse_iterator rbegin () noexcept`
Atvirkštinis pradžios iteratorius (rodo į paskutinį elementą).
- `const_reverse_iterator rbegin () const noexcept`
Konstantinis atvirkštinis pradžios iteratorius.
- `const_reverse_iterator crbegin () const noexcept`
Konstantinis atvirkštinis pradžios iteratorius (crbegin).
- `reverse_iterator rend () noexcept`
Atvirkštinis pabaigos iteratorius (rodo prieš pirmą elementą).
- `const_reverse_iterator rend () const noexcept`
Konstantinis atvirkštinis pabaigos iteratorius.
- `const_reverse_iterator crend () const noexcept`
Konstantinis atvirkštinis pabaigos iteratorius (crend).
- `bool empty () const noexcept`
Patikrina, ar masyvas tuščias.
- `size_type size () const noexcept`
Grąžina esamą elementų skaičių.
- `size_type capacity () const noexcept`
Grąžina dabartinę talpą (kiek elementų telpa be persikirstymo).
- `size_type max_size () const noexcept`
Grąžina teoriškai maksimalų galimą elementų skaičių.
- `void reserve (size_type n)`
Rezervuoja atmintį bent n elementams.
- `void shrink_to_fit ()`
Sumažina rezervuotą atmintį iki faktiškai naudojamos.
- `void resize (size_type sz, const value_type &val=value_type())`
Pakeičia masyvo dydį.
- `void push_back (const value_type &val)`
Prideda elemento kopiją masyvo gale.
- `void push_back (value_type &&val)`

- Perkelia elementą į masyvo galą.*

 - `template<typename... Args>`
`reference emplace_back (Args &&... args)`
Sukuria elementą tiesiogiai masyvo gale (be kopijavimo).
 - `template<std::ranges::input_range R>`
`void append_range (R &&r)`
Prideda diapazono elementus masyvo gale (C++23).
 - `void pop_back ()`
Pašalina paskutinį elementą.
 - `void clear () noexcept`
Sunaikina visus elementus, bet neatlaisvina rezervuotos atminties.
 - `iterator insert (const_iterator cpos, const value_type &val)`
Įterpia vieno elemento kopiją nurodytoje pozicijoje.
 - `iterator insert (const_iterator cpos, value_type &&val)`
Įterpia perkliamą elementą nurodytoje pozicijoje.
 - `iterator insert (const_iterator cpos, size_type n, const value_type &val)`
Įterpia n kopijų reikšmės val prieš nurodytą poziciją.
 - `template<std::input_iterator InputIt>`
`iterator insert (const_iterator cpos, InputIt first, InputIt last)`
Įterpia elementus iš iteratorių diapazono prieš nurodytą poziciją.
 - `iterator insert (const_iterator cpos, std::initializer_list< value_type > il)`
Įterpia inicializavimo sąrašo elementus prieš nurodytą poziciją.
 - `template<std::ranges::input_range R>`
`iterator insert_range (const_iterator cpos, R &&r)`
Įterpia diapazono elementus prieš nurodytą poziciją (C++23).
 - `template<typename... Args>`
`iterator emplace (const_iterator cpos, Args &&... args)`
Sukuria elementą tiesiogiai nurodytoje pozicijoje.
 - `iterator erase (const_iterator cpos)`
Pašalina elementą nurodytoje pozicijoje.
 - `iterator erase (const_iterator cfirst, const_iterator clast)`
Pašalina elementus iš diapazono [cfirst, clast).
 - `void swap (Vector &other) noexcept`
Keičia šio masyvo turinį su kito masyvo turiniu.
 - `bool operator== (const Vector &other) const`
Lygybės operatorius.
 - `auto operator<=> (const Vector &other) const`
Trijų kryptų palyginimo operatorius (C++20, <=>).
 - `bool operator!= (const Vector &other) const`
Nelygybės operatorius.
 - `bool operator< (const Vector &other) const`
Mažiau operatorius.
 - `bool operator<= (const Vector &other) const`
Mažiau arba lygu operatorius.
 - `bool operator> (const Vector &other) const`
Daugiau operatorius.
 - `bool operator>= (const Vector &other) const`
Daugiau arba lygu operatorius.

5.1.1 Smulkus aprašymas

```
template<VectorElement T>
class Vector< T >
```

Dinaminio masyvo šabloninė klasė.

Saugo elementus nuosekliame atminties bloke. Kai laisvos vietos nebelieka, masyvas automatiškai išplečiamas perkėlimo operacija į naują, didesnę bloką.

Template Parameters

<i>T</i>	Elementų tipas; turi tenkinti VectorElement sąvoką.
----------	---

5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.2.1 Vector() [1/5]

```
template<VectorElement T>
Vector< T >::Vector (
    size_type n,
    const T & val = T{}) [inline], [explicit]
```

Konstruktorius su pradiniu dydžiu ir reikšme.

Parametrai

<i>n</i>	Pradinių elementų skaičius.
<i>val</i>	Pradinė visų elementų reikšmė (numatytoji — <code>T{ }</code>).

5.1.2.2 Vector() [2/5]

```
template<VectorElement T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją kito [Vector](#) objekto.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

5.1.2.3 Vector() [3/5]

```
template<VectorElement T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

Perkėlimo konstruktorius.

Perimamas `other` turinys; po operacijos `other` tampa tuščias.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

5.1.2.4 Vector() [4/5]

```
template<VectorElement T>
template<std::input_iterator InputIt>
Vector< T >::Vector (
    InputIt first,
    InputIt last) [inline]
```

Konstruktorius iš iteratorių diapazono.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas.
----------------	------------------------------

Parametrai

<i>first</i>	Diapazono pradžia.
<i>last</i>	Diapazono pabaiga (neįskaičiuojama).

5.1.2.5 Vector() [5/5]

```
template<VectorElement T>
Vector< T >::Vector (
    std::initializer_list< T > il) [inline]
```

Konstruktorius iš inicializavimo sąrašo.

Parametrai

<i>il</i>	Inicializavimo sąrašas, pvz., {1, 2, 3}.
-----------	--

5.1.3 Metodų Dokumentacija

5.1.3.1 append_range()

```
template<VectorElement T>
template<std::ranges::input_range R>
void Vector< T >::append_range (
    R && r) [inline]
```

Prideda diapazono elementus masyvo gale (C++23).

Template Parameters

<i>R</i>	Įvesties diapazono tipas.
----------	---------------------------

Parametrai

<i>r</i>	Diapazonas, iš kurio paimami elementai.
----------	---

5.1.3.2 assign() [1/3]

```
template<VectorElement T>
template<std::input_iterator InputIt>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

Pakeičia turinį elementais iš iteratorių diapazono.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas.
----------------	------------------------------

Parametrai

<i>first</i>	Diapazono pradžia.
<i>last</i>	Diapazono pabaiga (neįskaičiuojama).

5.1.3.3 assign() [2/3]

```
template<VectorElement T>
void Vector< T >::assign (
    size_type n,
    const value_type & val) [inline]
```

Pakeičia turinį *n* kopijų reikšmės *val*.

Parametrai

<i>n</i>	Elementų skaičius.
----------	--------------------

<i>val</i>	Reikšmė, kuria pildoma.
------------	-------------------------

5.1.3.4 assign() [3/3]

```
template<VectorElement T>
void Vector< T >::assign (
    std::initializer_list< value_type > il) [inline]
```

Pakeičia turinį inicializavimo sąrašo elementais.

Parametrai

<i>il</i>	Inicializavimo sąrašas.
-----------	-------------------------

5.1.3.5 assign_range()

```
template<VectorElement T>
template<std::ranges::input_range R>
void Vector< T >::assign_range (
    R && r) [inline]
```

Pakeičia turinį diapazono elementais (C++23).

Template Parameters

<i>R</i>	Įvesties diapazono tipas.
----------	---------------------------

Parametrai

<i>r</i>	Diapazonas, iš kurio paimami elementai.
----------	---

5.1.3.6 at() [1/2]

```
template<VectorElement T>
reference Vector< T >::at (
    size_type n) [inline]
```

Prieiga prie elemento pagal indeksą su ribų tikrinimu.

Parametrai

<i>n</i>	Elemento indeksas.
----------	--------------------

Gražina

Nuoroda į elementą.

Išimtys

<i>std::out_of_range</i>	Jei $n \geq \text{size}()$.
--------------------------	------------------------------

5.1.3.7 at() [2/2]

```
template<VectorElement T>
const_reference Vector< T >::at (
    size_type n) const [inline]
```

Konstantinė prieiga prie elemento pagal indeksą su ribų tikrinimu.

Parametrai

<i>n</i>	Elemento indeksas.
----------	--------------------

Gražina

Konstantinė nuoroda į elementą.

Išimtys

<i>std::out_of_range</i>	Jei <i>n</i> $\geq$ <code>size()</code> .
--------------------------	---

5.1.3.8 back() [1/2]

```
template<VectorElement T>
reference Vector< T >::back () [inline]
```

Gražina nuorodą į paskutinį elementą.

Gražina

Nuoroda į paskutinį elementą.

5.1.3.9 back() [2/2]

```
template<VectorElement T>
const_reference Vector< T >::back () const [inline]
```

Gražina konstantinę nuorodą į paskutinį elementą.

Gražina

Konstantinė nuoroda į paskutinį elementą.

5.1.3.10 capacity()

```
template<VectorElement T>
size_type Vector< T >::capacity () const [inline], [nodiscard], [noexcept]
```

Gražina dabartinę talpą (kiek elementų telpa be persikirstymo).

Gražina

Talpa elementais.

5.1.3.11 clear()

```
template<VectorElement T>
void Vector< T >::clear () [inline], [noexcept]
```

Sunaikina visus elementus, bet neatlaisvina rezervuotos atminties.

Po operacijos `size() == 0`, tačiau `capacity()` lieka nepakitusi.

5.1.3.12 data() [1/2]

```
template<VectorElement T>
const value_type * Vector< T >::data () const [inline], [noexcept]
```

Konstantinė rodyklė į vidinę atmintį.

Gražina

Konstantinė rodyklė į pirmą elementą.

5.1.3.13 data() [2/2]

```
template<VectorElement T>
value_type * Vector< T >::data () [inline], [noexcept]
```

Tiesioginis prieigos prie vidinės atminties rodyklė.

Gražina

Rodyklė į pirmą elementą.

5.1.3.14 emplace()

```
template<VectorElement T>
template<typename... Args>
iterator Vector< T >::emplace (
    const_iterator cpos,
    Args &&... args) [inline]
```

Sukuria elementą tiesiogiai nurodytoje pozicijoje.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentų tipai.
-------------	----------------------------------

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią elementas sukuriamas.
-------------	---

<i>args</i>	Argumentai, perduodami T konstruktoriui.
-------------	--

Gražina

Iteratorius į sukurtą elementą.

5.1.3.15 `emplace_back()`

```
template<VectorElement T>
template<typename... Args>
reference Vector< T >::emplace_back (
    Args &&... args) [inline]
```

Sukuria elementą tiesiogiai masyvo gale (be kopijavimo).

Template Parameters

<i>Args</i>	Konstruktoriaus argumentų tipai.
-------------	----------------------------------

Parametrai

<i>args</i>	Argumentai, perduodami T konstruktoriui.
-------------	--

Gražina

Nuoroda į sukurtą elementą.

5.1.3.16 `empty()`

```
template<VectorElement T>
bool Vector< T >::empty () const [inline], [nodiscard], [noexcept]
```

Patikrina, ar masyvas tuščias.

Gražina

`true`, jei nėra elementų.

5.1.3.17 `erase()` [1/2]

```
template<VectorElement T>
iterator Vector< T >::erase (
    const_iterator cfirst,
    const_iterator clast) [inline]
```

Pašalina elementus iš diapazono [`cfirst`, `clast`).

Parametrai

<i>cfirst</i>	Iteratorius į pirmą šalinamą elementą.
---------------	--

<i>clast</i>	Iteratorius už paskutinio šalinamo elemento.
--------------	--

Gražina

Iteratorius į elementą, buvusį iškart po pašalintojo diapazono.

5.1.3.18 erase() [2/2]

```
template<VectorElement T>
iterator Vector< T >::erase (
    const_iterator cpos) [inline]
```

Pašalina elementą nurodytoje pozicijoje.

Parametrai

<i>cpos</i>	Iteratorius į šalinamą elementą.
-------------	----------------------------------

Gražina

Iteratorius į elementą, buvusį iškart po pašalintojo.

5.1.3.19 front() [1/2]

```
template<VectorElement T>
reference Vector< T >::front () [inline]
```

Gražina nuorodą į pirmą elementą.

Gražina

Nuoroda į pirmą elementą.

5.1.3.20 front() [2/2]

```
template<VectorElement T>
const_reference Vector< T >::front () const [inline]
```

Gražina konstantinę nuorodą į pirmą elementą.

Gražina

Konstantinė nuoroda į pirmą elementą.

5.1.3.21 get_allocator()

```
template<VectorElement T>
allocator_type Vector< T >::get_allocator () const [inline], [noexcept]
```

Gražina naudojamą atminties valdytojo kopiją.

Gražina

Atminties valdytojas.

5.1.3.22 insert() [1/5]

```
template<VectorElement T>
iterator Vector< T >::insert (
    const_iterator cpos,
    const value_type & val) [inline]
```

Įterpia vieno elemento kopiją nurodytoje pozicijoje.

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
<i>val</i>	Įterpiama reikšmė.

Gražina

Iteratorius į įterptą elementą.

5.1.3.23 insert() [2/5]

```
template<VectorElement T>
template<std::input_iterator InputIt>
iterator Vector< T >::insert (
    const_iterator cpos,
    InputIt first,
    InputIt last) [inline]
```

Įterpia elementus iš iteratorių diapazono prieš nurodytą poziciją.

Naudoja kopijavimo–keitimo strategiją stipriai išimčių garantijai.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas.
----------------	------------------------------

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
-------------	--

<i>first</i>	Įterpiamo diapazono pradžia.
<i>last</i>	Įterpiamo diapazono pabaiga (neįskaičiuojama).

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.24 insert() [3/5]

```
template<VectorElement T>
iterator Vector< T >::insert (
    const_iterator cpos,
    size_type n,
    const value_type & val) [inline]
```

Įterpia *n* kopijų reikšmės *val* prieš nurodytą poziciją.

Naudoja kopijavimo–keitimo strategiją stipriai išimčių garantijai.

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
<i>n</i>	Įterpiamų elementų skaičius.
<i>val</i>	Įterpiama reikšmė.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.25 insert() [4/5]

```
template<VectorElement T>
iterator Vector< T >::insert (
    const_iterator cpos,
    std::initializer_list< value_type > il) [inline]
```

Įterpia inicializavimo sąrašo elementus prieš nurodytą poziciją.

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
<i>il</i>	Inicializavimo sąrašas.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.26 insert() [5/5]

```
template<VectorElement T>
iterator Vector< T >::insert (
    const_iterator cpos,
    value_type && val) [inline]
```

Įterpia perkeliama elementą nurodytoje pozicijoje.

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
<i>val</i>	Perkeliamas reikšmė.

Gražina

Iteratorius į įterptą elementą.

5.1.3.27 insert_range()

```
template<VectorElement T>
template<std::ranges::input_range R>
iterator Vector< T >::insert_range (
    const_iterator cpos,
    R && r) [inline]
```

Įterpia diapazono elementus prieš nurodytą poziciją (C++23).

Template Parameters

<i>R</i>	Įvesties diapazono tipas.
----------	---------------------------

Parametrai

<i>cpos</i>	Iteratorius į poziciją, prieš kurią įterpiama.
<i>r</i>	Diapazonas, iš kurio paimami elementai.

Gražina

Iteratorius į pirmą įterptą elementą.

5.1.3.28 max_size()

```
template<VectorElement T>
size_type Vector< T >::max_size () const [inline], [nodiscard], [noexcept]
```

Gražina teoriškai maksimalų galimą elementų skaičių.

Gražina

Maksimalus skaičius (platformos riba).

5.1.3.29 operator std::vector< T >()

```
template<VectorElement T>
Vector< T >::operator std::vector< T > () const [inline]
```

Netiesioginis konvertavimas į `std::vector<T>`.

Sukuria kopiją visų elementų.

5.1.3.30 operator<=>()

```
template<VectorElement T>
auto Vector< T >::operator<=> (
    const Vector< T > & other) const [inline]
```

Trijų krypčių palyginimo operatorius (C++20, <=>).

Leksikografinė tvarka.

Parametrai

<i>other</i>	Lyginamas masyvas.
--------------	--------------------

Gražina

Palyginimo kategorija (`std::strong_ordering` arba silpnesnė, priklausomai nuo `T`).

5.1.3.31 operator=() [1/3]

```
template<VectorElement T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Atlieka gilią kopiją. Apsaugo nuo savęs priskyrimo.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Gražina

Nuoroda į šį objektą.

5.1.3.32 operator=() [2/3]

```
template<VectorElement T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > il) [inline]
```

Priskyrimo operatorius iš inicializavimo sąrašo.

Parametrai

<i>il</i>	Inicializavimo sąrašas.
-----------	-------------------------

Gražina

Nuoroda į šį objektą.

5.1.3.33 operator=() [3/3]

```
template<VectorElement T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

Perimamas *other* turinys; po operacijos *other* tampa tuščias.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

Gražina

Nuoroda į šį objektą.

5.1.3.34 operator==()

```
template<VectorElement T>
bool Vector< T >::operator== (
    const Vector< T > & other) const [inline]
```

Lygybės operatorius.

Parametrai

<i>other</i>	Lyginamas masyvas.
--------------	--------------------

Gražina

true, jei abu masyvai turi vienodus elementus ta pačia tvarka.

5.1.3.35 operator[]() [1/2]

```
template<VectorElement T>
reference Vector< T >::operator[] (
    size_type n) [inline]
```

Prieiga prie elemento pagal indeksą (be ribų tikrinimo).

Parametrai

<i>n</i>	Elemento indeksas.
----------	--------------------

Gražina

Nuoroda į elementą.

5.1.3.36 operator[]() [2/2]

```
template<VectorElement T>
const_reference Vector< T >::operator[] (
    size_type n) const [inline]
```

Konstantinė prieiga prie elemento pagal indeksą (be ribų tikrinimo).

Parametrai

<i>n</i>	Elemento indeksas.
----------	--------------------

Gražina

Konstantinė nuoroda į elementą.

5.1.3.37 pop_back()

```
template<VectorElement T>
void Vector< T >::pop_back () [inline]
```

Pašalina paskutinį elementą.

Nieko nevykdo, jei masyvas tuščias.

5.1.3.38 push_back() [1/2]

```
template<VectorElement T>
void Vector< T >::push_back (
    const value_type & val) [inline]
```

Prideda elemento kopiją masyvo gale.

Parametrai

<i>val</i>	Pridedama reikšmė.
------------	--------------------

5.1.3.39 push_back() [2/2]

```
template<VectorElement T>
void Vector< T >::push_back (
    value_type && val) [inline]
```

Perkelia elementą į masyvo galą.

Parametrai

<i>val</i>	Perkeliamas reikšmė.
------------	----------------------

5.1.3.40 reserve()

```
template<VectorElement T>
void Vector< T >::reserve (
    size_type n) [inline]
```

Rezervuoja atmintį bent *n* elementams.

Jei dabartinė talpa jau pakankama, nieko nevykdo.

Parametrai

<i>n</i>	Pageidaujama minimali talpa.
----------	------------------------------

5.1.3.41 resize()

```
template<VectorElement T>
void Vector< T >::resize (
    size_type sz,
    const value_type & val = value_type()) [inline]
```

Pakeičia masyvo dydį.

- Jei *sz* < `size()` — pertekliniai elementai sunaikinami.
- Jei *sz* > `size()` — nauji elementai inicializuojami reikšme *val*.

Parametrai

<i>sz</i>	Pageidaujamas naujas dydis.
<i>val</i>	Reikšmė naujiems elementams (numatytoji — <code>value_type()</code>).

5.1.3.42 shrink_to_fit()

```
template<VectorElement T>
void Vector< T >::shrink_to_fit () [inline]
```

Sumažina rezervuotą atmintį iki faktiškai naudojamą.

Talpa tampa lygi `size()`. Visi esami elementai perkeliama.

5.1.3.43 size()

```
template<VectorElement T>
size_type Vector< T >::size () const [inline], [nodiscard], [noexcept]
```

Grąžina esamų elementų skaičių.

Gražina

Elementų skaičius.

5.1.3.44 swap()

```
template<VectorElement T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

Keičia šio masyvo turinį su kito masyvo turiniu.

Operacija vykdoma be kopijų, O(1) laikinė sudėtingumas.

Parametrai

<i>other</i>	Masyvas, su kuriuo keičiamasi.
--------------	--------------------------------

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [Vector.h](#)

skyrius 6

Failo Dokumentacija

6.1 Vector.h Failo Nuoroda

Dinaminio masyvo šabloninė klasė, analogiška `std::vector`.

```
#include <algorithm>
#include <compare>
#include <concepts>
#include <initializer_list>
#include <iterator>
#include <limits>
#include <memory>
#include <span>
#include <stdexcept>
#include <type_traits>
#include <vector>
```

Klasės

- class [Vector< T >](#)

Dinaminio masyvo šabloninė klasė.

Concepts

- concept [VectorElement](#)

Sąvoka, reikalaujanti, kad tipas T būtų perkeliamas (`std::movable`).

Funkcijos

- template<[VectorElement](#) T>
void [swap](#) ([Vector](#)< T > &a, [Vector](#)< T > &b) noexcept
Nenarė funkcija keitimuisi dviem [Vector](#) objektais.
- template<[VectorElement](#) T, typename U>
[Vector](#)< T >::size_type [erase](#) ([Vector](#)< T > &v, const U &value)
Pašalina visus elementus, lygius value (C++20).
- template<[VectorElement](#) T, typename Pred>
[Vector](#)< T >::size_type [erase_if](#) ([Vector](#)< T > &v, Pred pred)
Pašalina visus elementus, tenkinančius predikato sąlygą (C++20).

6.1.1 Smulkus aprašymas

Dinaminio masyvo šabloninė klasė, analogiška `std::vector`.

Šiame faile apibrėžta `Vector<T>` klasė — pilnavertis dinaminis masyvas, suderinamas su C++20/23 standartais. Palaikomi:

- pilnas iteratorių rinkinys (tiesioginiai ir atvirkštiniai),
- `emplace_back`, `insert`, `erase` su stipria išimčių garantija,
- C++20 trijų krypčių palyginimo operatorius (`<=>`),
- C++23 operacijos `append_range`, `assign_range`, `insert_range`.

Template Parameters

<i>T</i>	Elementų tipas; turi tenkinti <code>VectorElement</code> sąvoką (t.y. būti perkeliamas).
----------	--

6.1.2 Funkcijos Dokumentacija

6.1.2.1 `erase()`

```
template<VectorElement T, typename U>
Vector< T >::size_type erase (
    Vector< T > & v,
    const U & value)
```

Pašalina visus elementus, lygius `value` (C++20).

Viduje naudoja `std::remove` + `Vector::erase`.

Template Parameters

<i>T</i>	Elementų tipas.
<i>U</i>	Lyginamos reikšmės tipas.

Parametrai

<i>v</i>	Masyvas, iš kurio šalinama.
<i>value</i>	Reikšmė, kurią atitinkantys elementai šalinami.

Gražina

Pašalintų elementų skaičius.

6.1.2.2 erase_if()

```
template<VectorElement T, typename Pred>
Vector< T >::size_type erase_if (
    Vector< T > & v,
    Pred pred)
```

Pašalina visus elementus, tenkinančius predikato sąlygą (C++20).

Viduje naudoja `std::remove_if` + `Vector::erase`.

Template Parameters

<i>T</i>	Elementų tipas.
----------	-----------------

<i>Pred</i>	Unarinio predikato tipas.
-------------	---------------------------

Parametrai

<i>v</i>	Masyvas, iš kurio šalinama.
<i>pred</i>	Predikatas; elementas šalinamas, jei <code>pred(elem)</code> grąžina <code>true</code> .

Gražina

Pašalintų elementų skaičius.

6.1.2.3 swap()

```
template<VectorElement T>
void swap (
    Vector< T > & a,
    Vector< T > & b) [noexcept]
```

Nenarė funkcija keitimuisi dviem `Vector` objektais.

Iškviečia `a.swap(b)`, $O(1)$ sudėtingumas.

Template Parameters

<i>T</i>	Elementų tipas.
----------	-----------------

Parametrai

<i>a</i>	Pirmas masyvas.
<i>b</i>	Antras masyvas.

6.2 Vector.h

[Eiti į šio failo dokumentaciją.](#)

```
00001
00014
00015 #ifndef VECTOR_H
00016 #define VECTOR_H
00017
00018 #include <algorithm>
00019 #include <compare>
00020 #include <concepts>
00021 #include <initializer_list>
00022 #include <iterator>
00023 #include <limits>
00024 #include <memory>
00025 #include <span>
00026 #include <stdexcept>
00027 #include <type_traits>
00028 #include <vector>
00029
00037 template <typename T>
00038 concept VectorElement = std::movable<T>;
00039
```

```

00048 template <VectorElement T>
00049 class Vector {
00050 public:
00051     // Tipų sinonimai
00052
00053     using value_type           = T;
00054     using allocator_type       = std::allocator<T>;
00055     using size_type            = std::size_t;
00056     using difference_type      = std::ptrdiff_t;
00057     using reference            = T&;
00058     using const_reference      = const T&;
00059     using pointer              = typename std::allocator_traits<std::allocator<T>::pointer;
00060     using const_pointer        = typename std::allocator_traits<std::allocator<T>::const_pointer;
00061     using iterator             = T*;
00062     using const_iterator       = const T*;
00063     using reverse_iterator     = std::reverse_iterator<iterator>;
00064     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00065
00066     // Konstruktoriai
00067
00068     Vector() { create(); }
00069
00070     explicit Vector(size_type n, const T& val = T{}) { create(n, val); }
00071
00072     Vector(const Vector& other) { create(other.begin(), other.end()); }
00073
00074     Vector(Vector&& other) noexcept
00075         : dat(other.dat), avail(other.avail), limit(other.limit)
00076     {
00077         other.dat = other.avail = other.limit = nullptr;
00078     }
00079
00080     template <std::input_iterator InputIt>
00081     Vector(InputIt first, InputIt last) { create(first, last); }
00082
00083     Vector(std::initializer_list<T> il) { create(il.begin(), il.end()); }
00084
00085     ~Vector() { uncreate(); }
00086
00087     // Priskyrimo operatoriai
00088
00089     Vector& operator=(const Vector& other) {
00090         if (this != &other) {
00091             uncreate();
00092             create(other.begin(), other.end());
00093         }
00094         return *this;
00095     }
00096
00097     Vector& operator=(Vector&& other) noexcept {
00098         if (this != &other) {
00099             uncreate();
00100             dat = other.dat;
00101             avail = other.avail;
00102             limit = other.limit;
00103             other.dat = other.avail = other.limit = nullptr;
00104         }
00105         return *this;
00106     }
00107
00108     Vector& operator=(std::initializer_list<T> il) {
00109         uncreate();
00110         create(il.begin(), il.end());
00111         return *this;
00112     }
00113
00114     operator std::vector<T>() const { return std::vector<T>(dat, avail); }
00115
00116     operator std::span<T>() { return std::span<T>(dat, avail); }
00117
00118     operator std::span<const T>() const { return std::span<const T>(dat, avail); }
00119
00120     // assign
00121
00122     template <std::input_iterator InputIt>
00123     void assign(InputIt first, InputIt last) {
00124         uncreate();
00125         create(first, last);
00126     }
00127
00128     void assign(size_type n, const value_type& val) {
00129         uncreate();
00130         create(n, val);
00131     }
00132
00133     void assign(std::initializer_list<value_type> il) {
00134         uncreate();

```

```

00236         create(il.begin(), il.end());
00237     }
00238
00245     template <std::ranges::input_range R>
00246     void assign_range(R&& r) {
00247         uncreate();
00248         create(std::ranges::begin(r), std::ranges::end(r));
00249     }
00250
00251     // get_allocator
00252
00258     allocator_type get_allocator() const noexcept { return alloc; }
00259
00260     // Prieiga prie elementų
00261
00268     reference operator[](size_type n) { return dat[n]; }
00269
00276     const_reference operator[](size_type n) const { return dat[n]; }
00277
00285     reference at(size_type n) {
00286         if (n >= size())
00287             throw std::out_of_range("Vector::at: index out of range");
00288         return dat[n];
00289     }
00290
00298     const_reference at(size_type n) const {
00299         if (n >= size())
00300             throw std::out_of_range("Vector::at: index out of range");
00301         return dat[n];
00302     }
00303
00308     reference front() { return dat[0]; }
00309
00314     const_reference front() const { return dat[0]; }
00315
00320     reference back() { return avail[-1]; }
00321
00326     const_reference back() const { return avail[-1]; }
00327
00332     value_type* data() noexcept { return dat; }
00333
00338     const value_type* data() const noexcept { return dat; }
00339
00340     // Iteratoriai
00341
00343     iterator begin() noexcept { return dat; }
00345     const_iterator begin() const noexcept { return dat; }
00347     const_iterator cbegin() const noexcept { return dat; }
00348
00350     iterator end() noexcept { return avail; }
00352     const_iterator end() const noexcept { return avail; }
00354     const_iterator cend() const noexcept { return avail; }
00355
00357     reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00359     const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00361     const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00362
00364     reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00366     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00368     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00369
00370     // Talpa
00371
00376     [[nodiscard]] bool empty() const noexcept { return dat == avail; }
00377
00382     [[nodiscard]] size_type size() const noexcept { return static_cast<size_type>(avail - dat); }
00383
00388     [[nodiscard]] size_type capacity() const noexcept { return static_cast<size_type>(limit - dat); }
00389
00394     [[nodiscard]] size_type max_size() const noexcept { return std::numeric_limits<size_type>::max(); }
00395 }
00396
00403     void reserve(size_type n) {
00404         if (n > capacity())
00405             reallocate(n);
00406     }
00407
00413     void shrink_to_fit() {
00414         if (limit != avail) {
00415             size_type sz = size();
00416             iterator new_dat = alloc.allocate(sz);
00417             std::uninitialized_move(dat, avail, new_dat);
00418             uncreate();
00419             dat = new_dat;
00420             avail = limit = dat + sz;
00421         }
00422     }

```

```

00423
00433 void resize(size_type sz, const value_type& val = value_type()) {
00434     if (sz < size()) {
00435         for (iterator it = dat + sz; it != avail; ++it)
00436             alloc_traits::destroy(alloc, it);
00437         avail = dat + sz;
00438     } else if (sz > size()) {
00439         if (sz > capacity())
00440             reallocate(sz);
00441         std::uninitialized_fill(avail, dat + sz, val);
00442         avail = dat + sz;
00443     }
00444 }
00445
00446 // Keitimo operacijos
00447
00453 void push_back(const value_type& val) {
00454     if (avail == limit) grow();
00455     alloc_traits::construct(alloc, avail++, val);
00456 }
00457
00463 void push_back(value_type&& val) {
00464     if (avail == limit) grow();
00465     alloc_traits::construct(alloc, avail++, std::move(val));
00466 }
00467
00475 template <typename... Args>
00476 reference emplace_back(Args&&... args) {
00477     if (avail == limit) grow();
00478     alloc_traits::construct(alloc, avail++, std::forward<Args>(args)...);
00479     return back();
00480 }
00481
00488 template <std::ranges::input_range R>
00489 void append_range(R&& r) {
00490     for (auto&& elem : r)
00491         push_back(std::forward<decltype(elem)>(elem));
00492 }
00493
00499 void pop_back() {
00500     if (avail != dat)
00501         alloc_traits::destroy(alloc, --avail);
00502 }
00503
00509 void clear() noexcept {
00510     for (iterator it = dat; it != avail; ++it)
00511         alloc_traits::destroy(alloc, it);
00512     avail = dat;
00513 }
00514
00522 iterator insert(const_iterator cpos, const value_type& val) {
00523     return insert(cpos, size_type{1}, val);
00524 }
00525
00533 iterator insert(const_iterator cpos, value_type&& val) {
00534     size_type index = static_cast<size_type>(cpos - dat);
00535     if (avail == limit) grow(size() + 1);
00536     iterator pos = dat + index;
00537     shift_right(pos, 1);
00538     alloc_traits::construct(alloc, pos, std::move(val));
00539     ++avail;
00540     return pos;
00541 }
00542
00553 iterator insert(const_iterator cpos, size_type n, const value_type& val) {
00554     if (n == 0) return const_cast<iterator>(cpos);
00555     size_type index = static_cast<size_type>(cpos - dat);
00556
00557     Vector tmp;
00558     tmp.reallocate(size() + n);
00559     std::uninitialized_copy(dat, dat + index, tmp.dat);
00560     std::uninitialized_fill(tmp.dat + index, tmp.dat + index + n, val);
00561     std::uninitialized_copy(dat + index, avail, tmp.dat + index + n);
00562     tmp.avail = tmp.dat + size() + n;
00563     swap(tmp);
00564
00565     return dat + index;
00566 }
00567
00579 template <std::input_iterator InputIt>
00580 iterator insert(const_iterator cpos, InputIt first, InputIt last) {
00581     size_type index = static_cast<size_type>(cpos - dat);
00582     size_type n = static_cast<size_type>(std::distance(first, last));
00583     if (n == 0) return dat + index;
00584
00585     Vector tmp;
00586     tmp.reallocate(size() + n);

```

```

00587         std::uninitialized_copy(dat, dat + index, tmp.dat);
00588         std::uninitialized_copy(first, last, tmp.dat + index);
00589         std::uninitialized_copy(dat + index, avail, tmp.dat + index + n);
00590         tmp.avail = tmp.dat + size() + n;
00591         swap(tmp);
00592
00593         return dat + index;
00594     }
00595
00603     iterator insert(const_iterator cpos, std::initializer_list<value_type> il) {
00604         return insert(cpos, il.begin(), il.end());
00605     }
00606
00615     template <std::ranges::input_range R>
00616     iterator insert_range(const_iterator cpos, R&& r) {
00617         size_type index = static_cast<size_type>(cpos - dat);
00618         size_type n      = static_cast<size_type>(std::ranges::distance(r));
00619         if (n == 0) return dat + index;
00620
00621         Vector tmp;
00622         tmp.reallocate(size() + n);
00623         std::uninitialized_copy(dat, dat + index, tmp.dat);
00624         std::uninitialized_copy(std::ranges::begin(r), std::ranges::end(r), tmp.dat + index);
00625         std::uninitialized_copy(dat + index, avail, tmp.dat + index + n);
00626         tmp.avail = tmp.dat + size() + n;
00627         swap(tmp);
00628
00629         return dat + index;
00630     }
00631
00640     template <typename... Args>
00641     iterator emplace(const_iterator cpos, Args&&... args) {
00642         size_type index = static_cast<size_type>(cpos - dat);
00643         if (avail == limit) grow(size() + 1);
00644         iterator pos = dat + index;
00645         shift_right(pos, 1);
00646         alloc_traits::construct(alloc, pos, std::forward<Args>(args)...);
00647         ++avail;
00648         return pos;
00649     }
00650
00657     iterator erase(const_iterator cpos) {
00658         iterator pos = const_cast<iterator>(cpos);
00659         std::move(pos + 1, avail, pos);
00660         alloc_traits::destroy(alloc, --avail);
00661         return pos;
00662     }
00663
00671     iterator erase(const_iterator cfirst, const_iterator clast) {
00672         iterator first = const_cast<iterator>(cfirst);
00673         iterator last = const_cast<iterator>(clast);
00674         iterator new_avail = std::move(last, avail, first);
00675         for (iterator it = new_avail; it != avail; ++it)
00676             alloc_traits::destroy(alloc, it);
00677         avail = new_avail;
00678         return first;
00679     }
00680
00688     void swap(Vector& other) noexcept {
00689         std::swap(dat, other.dat);
00690         std::swap(avail, other.avail);
00691         std::swap(limit, other.limit);
00692     }
00693
00694     // Palyginimo operatoriai
00695
00702     bool operator==(const Vector& other) const {
00703         return size() == other.size() && std::equal(begin(), end(), other.begin());
00704     }
00705
00715     auto operator<=(const Vector& other) const {
00716         return std::lexicographical_compare_three_way(
00717             begin(), end(), other.begin(), other.end());
00718     }
00719
00721     bool operator!=(const Vector& other) const { return !(*this == other); }
00723     bool operator< (const Vector& other) const { return (*this <= other) < 0; }
00725     bool operator<= (const Vector& other) const { return (*this <= other) <= 0; }
00727     bool operator> (const Vector& other) const { return (*this <= other) > 0; }
00729     bool operator>= (const Vector& other) const { return (*this <= other) >= 0; }
00730
00731 private:
00733     iterator      dat      = nullptr;
00735     iterator      avail    = nullptr;
00737     iterator      limit    = nullptr;
00739     std::allocator<T> alloc;
00740

```



```

00742     using alloc_traits = std::allocator_traits<std::allocator<T>>;
00743
00747     void create() { dat = avail = limit = nullptr; }
00748
00755     void create(size_type n, const T& val) {
00756         dat = alloc.allocate(n);
00757         limit = avail = dat + n;
00758         std::uninitialized_fill(dat, limit, val);
00759     }
00760
00768     template <typename InputIt>
00769     void create(InputIt first, InputIt last) {
00770         size_type n = static_cast<size_type>(std::distance(first, last));
00771         dat = alloc.allocate(n);
00772         limit = avail = std::uninitialized_copy(first, last, dat);
00773     }
00774
00780     void uncreate() {
00781         if (dat) {
00782             for (iterator it = avail; it != dat; )
00783                 alloc_traits::destroy(alloc, --it);
00784             alloc.deallocate(dat, static_cast<size_type>(limit - dat));
00785         }
00786         dat = avail = limit = nullptr;
00787     }
00788
00796     void reallocate(size_type new_capacity) {
00797         iterator new_dat = alloc.allocate(new_capacity);
00798         iterator new_avail = std::uninitialized_move(dat, avail, new_dat);
00799         uncreate();
00800         dat = new_dat;
00801         avail = new_avail;
00802         limit = dat + new_capacity;
00803     }
00804
00812     void grow(size_type min_capacity = 0) {
00813         size_type new_cap = std::max({ min_capacity, 2 * capacity(), size_type{1} });
00814         reallocate(new_cap);
00815     }
00816
00825     void shift_right(iterator pos, size_type n) {
00826         if (pos == avail) return;
00827         size_type tail = static_cast<size_type>(avail - pos);
00828         if (n >= tail) {
00829             // Visi elementai [pos, avail) patenka į neinicializuotą sritį
00830             std::uninitialized_move(pos, avail, pos + n);
00831         } else {
00832             // Paskutiniai n elementų patenka į neinicializuotą sritį
00833             std::uninitialized_move(avail - n, avail, avail);
00834             // Likę elementai stumiami inicializuotoje srityje
00835             std::move_backward(pos, avail - n, avail);
00836         }
00837     }
00838 };
00839
00840 // Laisvosios funkcijos
00841
00851 template <VectorElement T>
00852 void swap(Vector<T>& a, Vector<T>& b) noexcept { a.swap(b); }
00853
00865 template <VectorElement T, typename U>
00866 typename Vector<T>::size_type erase(Vector<T>& v, const U& value) {
00867     auto it = std::remove(v.begin(), v.end(), value);
00868     auto n = static_cast<typename Vector<T>::size_type>(v.end() - it);
00869     v.erase(it, v.end());
00870     return n;
00871 }
00872
00884 template <VectorElement T, typename Pred>
00885 typename Vector<T>::size_type erase_if(Vector<T>& v, Pred pred) {
00886     auto it = std::remove_if(v.begin(), v.end(), pred);
00887     auto n = static_cast<typename Vector<T>::size_type>(v.end() - it);
00888     v.erase(it, v.end());
00889     return n;
00890 }
00891
00892 #endif // VECTOR_H

```


Rodyklë

`append_range`
 `Vector< T >`, [15](#)

`assign`
 `Vector< T >`, [15](#), [16](#)

`assign_range`
 `Vector< T >`, [16](#)

`at`
 `Vector< T >`, [16](#)

`back`
 `Vector< T >`, [17](#)

`capacity`
 `Vector< T >`, [17](#)

`clear`
 `Vector< T >`, [17](#)

`data`
 `Vector< T >`, [18](#)

`emplace`
 `Vector< T >`, [18](#)

`emplace_back`
 `Vector< T >`, [19](#)

`empty`
 `Vector< T >`, [19](#)

`erase`
 `Vector< T >`, [19](#), [20](#)
 `Vector.h`, [30](#)

`erase_if`
 `Vector.h`, [30](#)

`front`
 `Vector< T >`, [20](#)

`get_allocator`
 `Vector< T >`, [20](#)

`insert`
 `Vector< T >`, [21](#), [22](#)

`insert_range`
 `Vector< T >`, [23](#)

`max_size`
 `Vector< T >`, [23](#)

`operator std::vector< T >`
 `Vector< T >`, [23](#)

`operator<=>`
 `Vector< T >`, [24](#)

`operator=`
 `Vector< T >`, [24](#), [25](#)

`operator==`
 `Vector< T >`, [25](#)

`operator[]`
 `Vector< T >`, [25](#), [26](#)

`pop_back`
 `Vector< T >`, [26](#)

`push_back`
 `Vector< T >`, [26](#)

`reserve`
 `Vector< T >`, [27](#)

`resize`
 `Vector< T >`, [27](#)

`shrink_to_fit`
 `Vector< T >`, [27](#)

`size`
 `Vector< T >`, [28](#)

`swap`
 `Vector< T >`, [28](#)
 `Vector.h`, [32](#)

`Vector`
 `Vector< T >`, [13](#), [14](#)

`Vector< T >`, [9](#)
 `append_range`, [15](#)
 `assign`, [15](#), [16](#)
 `assign_range`, [16](#)
 `at`, [16](#)
 `back`, [17](#)
 `capacity`, [17](#)
 `clear`, [17](#)
 `data`, [18](#)
 `emplace`, [18](#)
 `emplace_back`, [19](#)
 `empty`, [19](#)
 `erase`, [19](#), [20](#)
 `front`, [20](#)
 `get_allocator`, [20](#)
 `insert`, [21](#), [22](#)
 `insert_range`, [23](#)
 `max_size`, [23](#)
 `operator std::vector< T >`, [23](#)
 `operator<=>`, [24](#)
 `operator=`, [24](#), [25](#)
 `operator==`, [25](#)
 `operator[]`, [25](#), [26](#)
 `pop_back`, [26](#)

- push_back, [26](#)
- reserve, [27](#)
- resize, [27](#)
- shrink_to_fit, [27](#)
- size, [28](#)
- swap, [28](#)
- Vector, [13](#), [14](#)
- Vector.h, [29](#)
 - erase, [30](#)
 - erase_if, [30](#)
 - swap, [32](#)
- VectorElement, [7](#)